# Microservicio de Priorización de Rutas de Atención en Salud para Población Rural Dispersa de Cundinamarca

Kevin Alexander Reyes Hurtado, Jennifer Tatiana Lopez Coy, Gabriel Santiago Contreras Martinez
*Programa de Ingeniería de Sistemas y Computación — Ciclo de Investigación*
*Universidad de Cundinamarca, Colombia*
kareyes@ucundinamarca.edu.co, jtatianalopez@ucundinamarca.edu.co, gsantiagomartinez@ucundinamarca.edu.co

***Resumen*—Las dificultades de movilización hacia los centros de salud en las zonas rurales de Cundinamarca generan barreras de acceso y tiempos de respuesta prolongados. Este documento presenta la arquitectura y las decisiones técnicas de un sistema de microservicios que, a partir de veredas reales obtenidas de datos abiertos (DANE/POT) y de un índice de vulnerabilidad simulado precargado en JSON, calcula una ruta priorizada para que un equipo de salud visite cinco veredas en una jornada. La necesidad de atención la estima un modelo de Gradient Boosting, elegido por validación cruzada frente a regresión lineal y Random Forest ($R^2$ = 0,95), y un K-Means asigna perfiles descriptivos de necesidad. La selección usa un heurístico multicriterio transparente (necesidad, cercanía por carretera y rotación equitativa) y el orden se obtiene con vecino más cercano y mejora local 2-opt sobre distancias reales de OSRM. La seguridad se basa en OAuth 2.0 (código de autorización con PKCE y credenciales de cliente) con tokens JWT RS256, y un registro de auditoría encadenado con SHA-256 permite reconstruir y reproducir cada ruta con sus parámetros. Se discuten las consideraciones éticas para evitar la estigmatización de las comunidades y las limitaciones del modelo con datos simulados. Treinta pruebas automatizadas verifican los modelos, el algoritmo, la autenticación y la integridad de la auditoría.**

***Términos de indexación*—Aprendizaje automático, auditoría, ética algorítmica, microservicios, OAuth 2.0, problema del viajante, salud rural, sistemas de apoyo a la decisión.**

## I. INTRODUCCIÓN

En las provincias de Cundinamarca, buena parte de la población rural vive en veredas dispersas conectadas por vías terciarias, donde un desplazamiento de pocos kilómetros puede tomar más de una hora. Los equipos extramurales de atención primaria deben decidir cada día qué veredas visitar y en qué orden, normalmente con información incompleta y sin herramientas de apoyo. Esta situación se relaciona directamente con el Objetivo de Desarrollo Sostenible 3, meta 3.8 de cobertura sanitaria universal [1], y con la Ruta Integral de Atención para la Promoción y Mantenimiento de la Salud, que contempla la atención en el entorno hogar [2].

El reto planteado en la Maratón de Programación con Impacto Social (Ciclo de Investigación) consiste en construir un microservicio que priorice las visitas domiciliarias de un equipo de salud y que demuestre una arquitectura segura, auditable y éticamente consciente. Las contribuciones de este trabajo son: (i) una arquitectura de dos microservicios con un servidor de autorización OAuth 2.0 propio; (ii) la integración de tres fuentes públicas de datos geográficos y de enrutamiento; (iii) un algoritmo de priorización explicable y determinista; (iv) un registro de auditoría a prueba de alteraciones que permite reproducir cualquier ruta, y (v) un análisis ético y de limitaciones del modelo.

## II. REQUERIMIENTOS Y ALCANCE

La Tabla I resume cómo se atiende cada requerimiento del reto y de las especificaciones del ciclo. Se indica con honestidad lo que quedó parcial, para que el jurado pueda contrastarlo con la demostración.

TABLA I
TRAZABILIDAD DE REQUERIMIENTOS

| Requerimiento | Solución implementada | Estado |
|---|---|---|
| 10–15 veredas con coordenadas e índice en JSON | Veredas reales vía API; índice simulado en JSON (2 584 veredas); el usuario elige 5–15 candidatas | Cumple |
| Ruta para 5 veredas en una jornada | POST /api/rutas: orden de visita, km, tiempo y advertencia de jornada > 8 h | Cumple |
| Algoritmo de optimización simple | Puntaje multicriterio + vecino más cercano + 2-opt | Cumple |
| IA como componente central | Gradient Boosting estima la necesidad; K-Means asigna perfiles (proceso KDD) | Cumple |
| OAuth 2.0 | Código de autorización + PKCE y client credentials; JWT RS256 | Cumple |
| Auditoría reconstruible | Registro encadenado SHA-256 con parámetros, entradas y resultado; reproducción | Cumple |
| Consideraciones éticas y limitaciones | Secciones VIII y IX; documento de uso de IA | Cumple |
| Microservicios y múltiples APIs | auth-service + rutas-service; Esri/DANE, Gobernación, OSRM, OSM | Cumple |
| PostgreSQL / Redis | PostgreSQL gestionado en la nube (auditoría y caché); SQLite en local | Cumple |
| Nube con CI/CD | GitHub Actions (pruebas + auditoría de dependencias) y despliegue continuo en Render (HTTPS) | Cumple |

Por delimitación del reto no se integra historia clínica ni GPS en tiempo real. El sistema no almacena datos de pacientes: su unidad de análisis es la vereda.

## III. ARQUITECTURA DEL SISTEMA

La Fig. 1 muestra la arquitectura. Se separó el servidor de autorización (*auth-service*) del servidor de recursos (*rutas-service*), de modo que este último nunca conoce contraseñas ni secretos: solo valida la firma de los tokens con la llave pública publicada en el endpoint JWKS. Ambos servicios están escritos en Python 3.11 con FastAPI, que valida automáticamente las entradas con Pydantic y genera la especificación OpenAPI en `/docs`. El frontend es una aplicación de una sola página (HTML, JavaScript y Leaflet) servida por rutas-service, que actúa como cliente público OAuth 2.0.

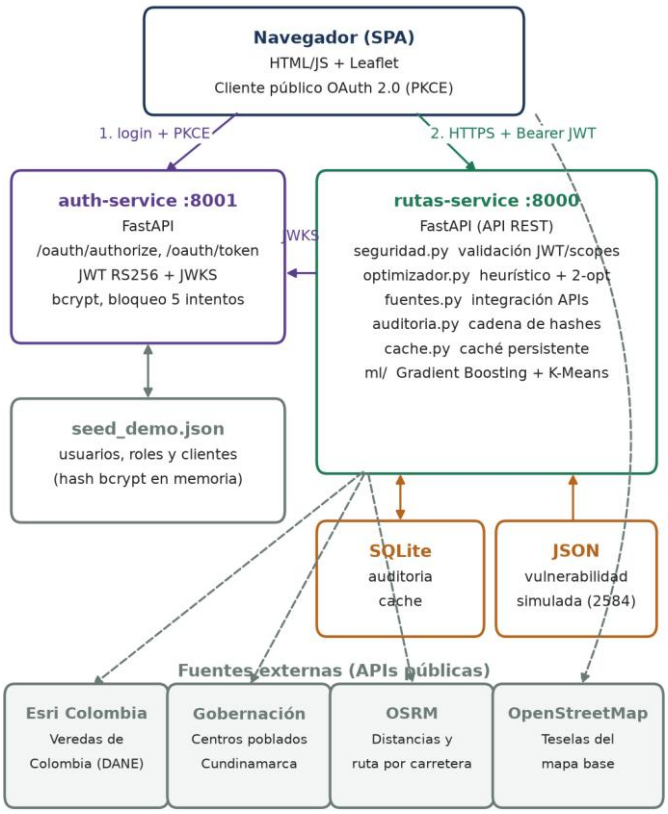



Fig. 1. Arquitectura de microservicios. Las líneas discontinuas son consultas a APIs públicas externas, todas con caché persistente y plan de contingencia.

TABLA II
ENDPOINTS PRINCIPALES

| Método y ruta | Permiso (scope) | Función |
|---|---|---|
| GET /oauth/authorize | — | Inicio de sesión (PKCE S256) |
| POST /oauth/token | — | Emite JWT (code o client credentials) |
| GET /.well-known/jwks.json | — | Llave pública RS256 |
| GET /api/municipios | veredas:leer | 116 municipios de Cundinamarca |
| GET /api/municipios/{cod}/veredas | veredas:leer | Polígonos, centroides y cabecera |
| GET /api/modelo | veredas:leer | Métricas del modelo de ML |
| POST /api/rutas | rutas:generar | Ruta priorizada y explicada |
| GET /api/auditoria[/{id}] | auditoria:leer | Consulta de registros |
| GET /api/auditoria/verificar | auditoria:leer | Verifica la cadena de hashes |
| POST /api/auditoria/{id}/reproducir | auditoria:leer | Re-ejecuta y compara |

### *A. Decisiones técnicas*

- *Python y FastAPI:* curva de aprendizaje baja, tipado con Pydantic y documentación OpenAPI automática; adecuado a las 2,5 horas de desarrollo.
- *RS256 en lugar de HS256:* la firma asimétrica permite agregar más servicios de recursos sin distribuir secretos.
- *Caché persistente con contingencia:* si una API externa falla se usa la última respuesta guardada; si OSRM no responde, se estima la distancia como haversine × 1,4.
- *Algoritmo como función pura:* las mismas entradas producen la misma salida, condición necesaria para que la auditoría pueda reproducir cualquier ruta.

## IV. INTEGRACIÓN DE FUENTES DE DATOS

Se evaluaron varias fuentes. El portal datos.gov.co solo ofrece conjuntos de veredas por municipio (p. ej., Chía), y el geoportal de la Superintendencia de Servicios Públicos no respondió durante las pruebas. Se seleccionaron:

- *Veredas de Colombia* [3]: capa ArcGIS REST con base DANE/POT; 2 584 veredas en Cundinamarca, con código de vereda (8 dígitos), municipio, área y polígono. El servicio calcula el centroide del anillo de mayor área.
- *Mapa Veredal de Cundinamarca* [4]: centros poblados de la Gobernación, usados para ubicar la cabecera municipal como punto de partida del equipo.
- *OSRM* [5]: servicio de enrutamiento sobre OpenStreetMap. El método table entrega la matriz de distancias y tiempos por carretera, y el método route el trazado que se dibuja en el mapa.
- *Teselas de OpenStreetMap* [6]: mapa base. Su política de uso exige el encabezado Referer, por lo que se configuró strict-origin-when-cross-origin (solo se envía el origen).

El código de municipio se valida con la expresión regular `^25\d{3}$` antes de construir la consulta a ArcGIS, lo que impide inyectar cláusulas en el parámetro where.

## V. MODELO DE DATOS Y BASE DE DATOS

La Fig. 2 presenta el modelo de datos. La persistencia propia del sistema tiene dos tablas: *auditoria*, solo de anexar, y *cache*, clave-valor con expiración. Las veredas son entidades externas consultadas a la API y guardadas en caché; el índice de vulnerabilidad es un JSON precargado relacionado 1:1 con la vereda por su código DANE, y los usuarios y clientes OAuth se cargan desde un archivo semilla y solo se conservan como hash bcrypt en memoria.

Cada registro de auditoría guarda el instante (UTC), el evento (RUTA_GENERADA, RUTA_REPRODUCIDA, ACCESO_DENEGADO, TOKEN_INVALIDO), el sujeto y el cliente del token, los roles, la IP, los parámetros de la solicitud, una copia de las entradas (base, candidatas con su índice, matriz de distancias y conteo de visitas recientes) y el resultado. El campo *hash* es el SHA-256 de la representación JSON canónica del registro, incluido el *hash_anterior*. Dos triggers impiden UPDATE y DELETE; si alguien con acceso al archivo los elimina y modifica un registro, el endpoint de verificación detecta el primer registro alterado.

*Motor de base de datos.* La capa db.py usa PostgreSQL cuando existe la variable DATABASE_URL (despliegue en la nube) y SQLite en local, con consultas siempre parametrizadas. En la nube, Render aprovisiona un PostgreSQL gestionado en la misma red privada que el servicio y sin acceso desde internet (lista de IP vacía); la cadena de conexión se inyecta como variable de entorno. El esquema (`rutas_service/db/schema_postgres.sql`) guarda los campos JSON como TEXT a propósito: el hash se calcula sobre el texto canónico, y JSONB lo normalizaría

rompiendo la verificación. Un trigger por sentencia bloquea UPDATE, DELETE y TRUNCATE, y cada escritura toma un bloqueo de tabla para que dos instancias no encadenen registros al mismo anterior. La caché de APIs externas usa la misma base con expiración (función equivalente a Redis). Tres pruebas verifican este comportamiento contra un PostgreSQL 16 real.

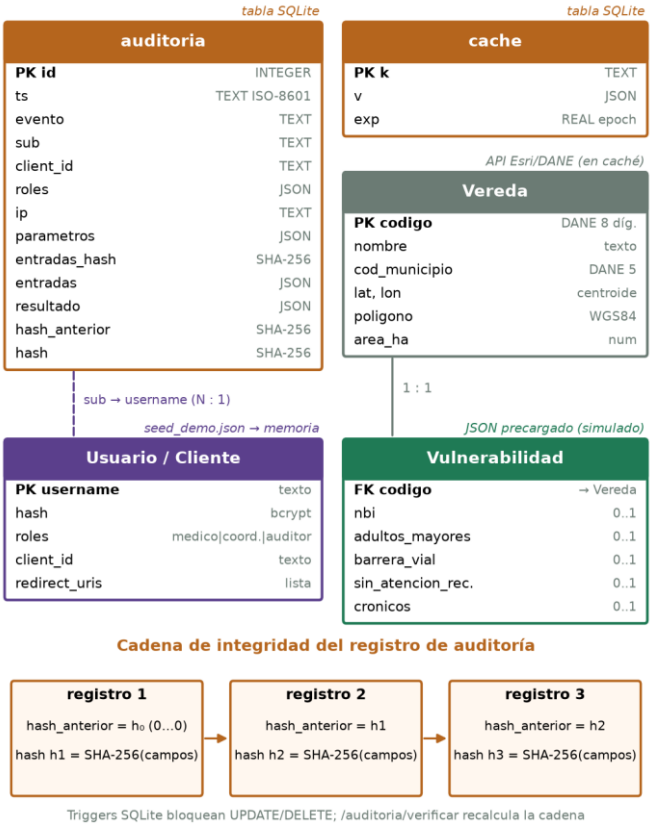



Fig. 2. Modelo de datos. Arriba, tablas SQLite; en el centro, entidades externas o precargadas; abajo, encadenamiento de hashes que hace evidente cualquier alteración del registro de auditoría.

## VI. ALGORITMO DE PRIORIZACIÓN

### *A. Modelo de aprendizaje automático (KDD)*

Siguiendo el proceso KDD (selección, preprocesamiento, transformación, minería e interpretación), se construyó un conjunto con las 2 584 veredas y cinco características de necesidad: NBI, población adulta mayor, barrera vial, tiempo sin atención y carga crónica. Como no existe una variable objetivo oficial a nivel veredal, se generó una necesidad observada sintética con interacciones no lineales (p. ej., adulto mayor × barrera vial) y ruido. Se compararon tres regresores con validación cruzada de 5 pliegues sobre el 80 % de los datos y se evaluó en el 20 % restante (Tabla III). Se eligió Gradient Boosting por el mayor $R^2$ en validación cruzada; supera a la fórmula de pesos fijos usada inicialmente. La importancia por permutación ordena las variables así: barrera vial (0,59), NBI (0,51), adultos mayores (0,36), crónicos (0,19) y tiempo sin atención (0,12).

TABLA III
SELECCIÓN DEL MODELO DE NECESIDAD

| Modelo | $R^2$ CV | $R^2$ prueba | MAE |
|---|---|---|---|
| Gradient Boosting (elegido) | 0,938 ± 0,007 | 0,953 | 0,028 |
| Regresión lineal | 0,926 ± 0,006 | 0,933 | 0,033 |
| Random Forest | 0,903 ± 0,013 | 0,915 | 0,036 |
| Fórmula de pesos fijos | — | 0,899 | 0,040 |

Además, un K-Means sobre las características estandarizadas (k = 5, elegido por silueta entre 3 y 6) agrupa las veredas en perfiles descriptivos, como "énfasis en barreras de acceso vial" o "énfasis en población adulta mayor". El perfil orienta el tipo de equipo o insumos de la visita y reemplaza etiquetas de ranking por una descripción de la necesidad.

### *B. Selección y orden de visita*

Sea C el conjunto de veredas candidatas (5 a 15), v$i$ ∈ [0,1] el índice de vulnerabilidad de la vereda i estimado por el modelo, d$i$ la distancia por carretera ida y vuelta desde la cabecera (promedio de ambos sentidos) y r$i$ el número de veces que la vereda fue programada en los últimos 30 días según la auditoría. El puntaje es:

$$p_i = \alpha \cdot v_i + (1 - \alpha) \cdot (1 - d_i / d_max) - \beta \cdot min(r_i, 3)/3 \quad (1)$$

donde α ∈ [0,1] es ajustable por el usuario (por defecto 0,6) y β = 0,1 cuando se activa la rotación equitativa. Se seleccionan las N veredas de mayor puntaje (por defecto N = 5; empates por código). El orden de visita resuelve un problema del viajante con regreso a la base:

$$min \sum D(\pi_k, \pi_{k+1}), \quad \pi_0 = \pi_N+1 = base \quad (2)$$

La solución inicial se construye con vecino más cercano [11] y se mejora con 2-opt [12], que invierte segmentos de la ruta mientras se reduzca la distancia total. Con N ≤ 8 el costo es despreciable ($O(N^2)$ por iteración). En las pruebas, la ruta obtenida queda a lo sumo 10 % por encima del óptimo calculado por fuerza bruta. El flujo completo se muestra en la Fig. 3.

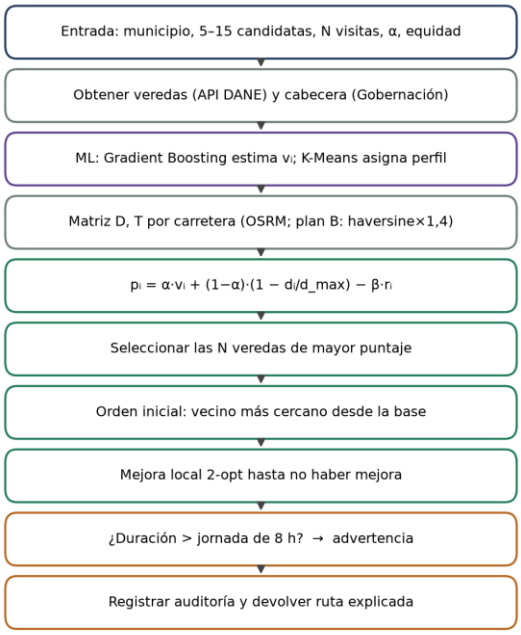



Fig. 3. Flujo del algoritmo de priorización y ordenamiento.

La respuesta incluye, para cada vereda, el aporte de la necesidad, de la cercanía y del ajuste por rotación, y el componente del índice con mayor peso, expresado en lenguaje neutral (p. ej., "barreras de acceso vial"). También se listan las candidatas no seleccionadas con su motivo. El tiempo total suma desplazamientos (OSRM) y el tiempo de atención por vereda (60 min por defecto); si supera 8 h se emite una advertencia.

## VII. SEGURIDAD

La Fig. 4 muestra el flujo de autenticación y generación de una ruta. El frontend usa el flujo de código de autorización [7] con PKCE S256 [8], recomendado para clientes públicos [10]; las integraciones entre sistemas usan client credentials con secreto almacenado como hash bcrypt. Los códigos de autorización expiran a los 60 s, son de un solo uso y solo se redirigen a URIs registradas. Los access tokens son JWT [9] firmados con RS256, válidos por 15 min, y rutas-service exige iss, aud, exp, iat y sub y solo acepta el algoritmo RS256 (rechaza tokens sin firma).

TABLA IV
ROLES Y PERMISOS (MÍNIMO PRIVILEGIO)

| Rol | Scopes |
|---|---|
| médico (equipo extramural) | veredas:leer, rutas:generar |
| coordinador APS | veredas:leer, rutas:generar, auditoria:leer |
| auditor | veredas:leer, auditoria:leer |
| sistema (client credentials) | rutas:generar |

TABLA V
CONTROLES FRENTE A OWASP TOP 10:2021

| Riesgo | Control implementado |
|---|---|
| A01 Control de acceso | Scopes por endpoint; 403 auditado; CORS restringido |
| A02 Fallas criptográficas | RS256 2048 bits; bcrypt; SHA-256 en auditoría; TLS en despliegue |
| A03 Inyección | Pydantic + regex en códigos DANE; consultas SQL parametrizadas; escape HTML |
| A04 Diseño inseguro | Servidor de autorización separado; PKCE obligatorio; mínimo privilegio |
| A05 Configuración | CSP, X-Frame-Options, nosniff, Referrer-Policy, Cache-Control no-store |
| A07 Autenticación | Bloqueo tras 5 intentos; mensaje genérico; comparación de tiempo constante |
| A08 Integridad | JWT firmado; cadena de hashes; SRI en Leaflet |
| A09 Registro y monitoreo | Auditoría de rutas, accesos denegados y tokens inválidos |

*Pruebas de penetración básicas.* Las pruebas automatizadas atacan el sistema con tokens firmados por otra llave, tokens expirados, tokens con alg = none, intentos de escalamiento de privilegios (auditor generando rutas, médico leyendo la auditoría), inyección en el código de municipio, reutilización de códigos de autorización, verificadores PKCE incorrectos, URIs de redirección no registradas, fuerza bruta de contraseñas y alteración directa de la base de datos. Todas son rechazadas o detectadas.

*Protección de datos (Ley 1581 de 2012).* Por minimización [14], el sistema no trata datos de salud de personas: opera sobre veredas. Los únicos datos personales son la identidad del usuario y su IP en la auditoría, con finalidad de trazabilidad. En producción se debe definir la política de retención, informar a los usuarios y cifrar el tránsito con TLS.

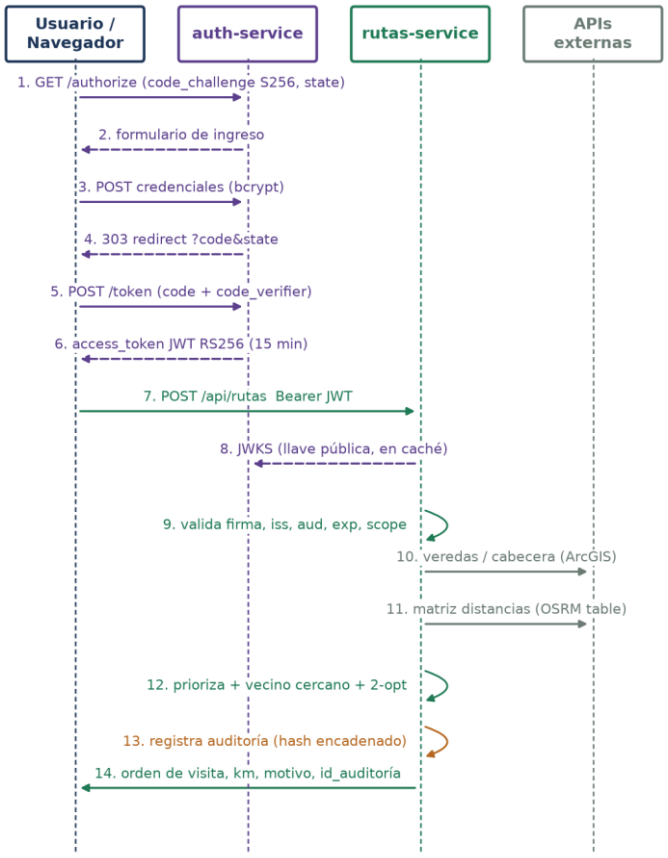



Fig. 4. Diagrama de secuencia: autenticación OAuth 2.0 con PKCE y generación auditada de una ruta.

*Configuración de producción.* Con ENTORNO = produccion los servicios no arrancan si las URLs no usan

HTTPS, agregan la cabecera HSTS, limitan CORS y las URIs de redirección a la URL pública del frontend y deshabilitan las credenciales de demostración publicadas en el repositorio: las contraseñas (mínimo 12 caracteres) y el secreto del cliente se inyectan como variables de entorno del proveedor de nube, nunca en el código. El despliegue se define como infraestructura como código (render.yaml) y tres pruebas verifican estas reglas.

*Despliegue.* El sistema está desplegado en Render: frontend y API en https://maraton-rutas-api.onrender.com y servidor de autorización en https://maraton-rutas-auth.onrender.com, con PostgreSQL gestionado. Cada cambio en la rama main dispara la construcción, ejecuta las pruebas y solo publica si todas pasan; se verificó HSTS, CSP, redirección de HTTP a HTTPS y rechazo (401) de peticiones sin token.

## VIII. CONSIDERACIONES ÉTICAS

Un índice de vulnerabilidad puede convertirse en una etiqueta que estigmatice a una comunidad ("la vereda más pobre") o en un mecanismo que excluya sistemáticamente a ciertas veredas. Siguiendo el Marco Ético para la IA en Colombia [15], se tomaron estas medidas:

- *Lenguaje de necesidad, no de déficit:* el índice se presenta como "necesidad de atención", y las explicaciones nombran factores estructurales (vías, cobertura) y no atributos de las personas.
- *Minimización:* el listado de veredas no expone el índice ni un ranking público; solo aparece, explicado, en la ruta generada por un usuario autorizado.
- *Sugerencia neutral:* las candidatas por defecto son las 15 más cercanas a la cabecera, no las más vulnerables.
- *Perfiles, no rankings:* el K-Means describe el tipo de necesidad de cada vereda en lugar de ordenarlas.
- *Equidad:* el ajuste por rotación evita que una vereda sea visitada siempre o nunca.
- *Transparencia y control humano:* pesos, fórmula y α son visibles; la ruta es una sugerencia y el aviso lo indica.
- *Rendición de cuentas:* cada ruta queda auditada con quién la pidió y con qué parámetros, y puede reproducirse.

Persisten riesgos: la cobertura de OpenStreetMap en vías rurales es menor que en zonas urbanas, lo que puede sobrestimar distancias a veredas remotas y penalizarlas por cercanía; un α bajo favorece a las veredas cercanas a la cabecera. Por ello α debe definirse con el equipo de salud y la comunidad, no solo por el desarrollador.

## IX. LIMITACIONES DEL MODELO

- El índice de vulnerabilidad es simulado (componentes beta deterministas por código); sus pesos no fueron validados por expertos en salud pública y no representan ninguna comunidad real.
- El modelo se entrenó con una variable objetivo sintética: sus métricas demuestran el proceso, no su validez real; la silueta del K-Means (0,15) indica perfiles poco separados.
- El centroide del polígono no coincide necesariamente con el lugar donde vive la población, sobre todo en veredas extensas.
- Los límites veredales provienen de POT con vigencias de alrededor de 2010; pueden estar desactualizados.
- OSRM no considera estado de la vía, clima, derrumbes, tramos a pie o fluviales, ni el tipo de vehículo; además, el servidor público de demostración no es apto para producción.
- Modela un solo equipo, una jornada de 8 h y tiempo de atención uniforme; no considera ventanas horarias ni capacidad.
- Los códigos de autorización y los contadores de intentos están en memoria (no distribuidos); la base gratuita de PostgreSQL tiene capacidad y vigencia limitadas.

## X. PRUEBAS Y RESULTADOS

TABLA VI
PRUEBAS AUTOMATIZADAS (PYTEST, 30/30 APROBADAS)

| Grupo | N.º | Qué verifican |
|---|---|---|
| Algoritmo | 6 | Selección de N, retorno a base, determinismo, extremos de α, cercanía al óptimo, equidad, jornada |
| OAuth 2.0 | 6 | Flujo PKCE, verificador incorrecto, código de un solo uso, redirect URI, bloqueo, client credentials, JWKS |
| Modelo de ML | 4 | Métricas, superioridad sobre la fórmula, rango, determinismo, monotonía |
| PostgreSQL | 3 | Cadena de hashes, bloqueo de UPDATE/DELETE/TRUNCATE y caché sobre PostgreSQL 16 |
| Producción | 3 | HTTPS obligatorio, credenciales demo deshabilitadas, contraseña mínima |
| API y seguridad | 8 | 401/403, llave ajena, expiración, alg none, validación 422, auditoría, reproducción, alteración |

En la demostración con el municipio de Pasca (28 veredas, 15 candidatas) la ruta de 5 veredas sumó 27,5 km por carretera y 6,3 h de jornada con 60 min de atención por vereda. Una segunda ejecución activó el ajuste de rotación y cambió la selección (30,1 km), lo que confirma el efecto de equidad. La primera consulta tarda unos 7 s por las APIs externas; las siguientes se resuelven desde la caché. La reproducción desde la auditoría devolvió el mismo orden y la misma distancia, y la verificación de la cadena reportó integridad.

## XI. IMPACTO Y SOSTENIBILIDAD

TABLA VII
INDICADORES PROPUESTOS

| Indicador | Medición |
|---|---|
| Cobertura de veredas prioritarias | % de veredas con necesidad alta visitadas en el mes |
| Eficiencia de desplazamiento | km y horas de viaje por vereda atendida |
| Equidad territorial | Coeficiente de Gini de visitas por vereda; días máximos sin visita |
| Cumplimiento de jornada | % de rutas sin advertencia de exceso de 8 h |
| Usabilidad (SUS) | Puntaje SUS ≥ 68 con equipos extramurales |
| HEART | Satisfacción, uso semanal, adopción por equipos, retención y % de rutas aceptadas sin cambios |

La usabilidad se medirá con la escala SUS [16] y el marco HEART [17] en una prueba piloto. En lo técnico, la solución usa estándares abiertos (OAuth 2.0, OpenAPI, GeoJSON) y software libre, y la capa de veredas cubre todo el país: escalar a otro departamento solo requiere cambiar el código DANE. En lo económico, las fuentes son gratuitas; en producción se

recomienda una instancia propia de OSRM con el extracto de Colombia, PostgreSQL gestionado y una máquina virtual pequeña, costos marginales frente al combustible y las horas de equipo que se ahorran. El pipeline de GitHub Actions ejecuta las pruebas y una auditoría de dependencias en cada cambio.

## XII. CONCLUSIONES Y TRABAJO FUTURO

Se construyó un sistema funcional que combina veredas reales, distancias por carretera y un índice simulado para sugerir rutas de atención explicables, seguras y auditables. El modelo de aprendizaje estima la necesidad y el heurístico transparente decide la ruta, de modo que cada elección sigue siendo explicable en un contexto de salud. Como trabajo futuro se propone construir el índice con fuentes oficiales (CNPV 2018, SISBÉN, RIPS), validarlo con la Secretaría de Salud y las comunidades, extender el modelo a varios equipos con ventanas de tiempo (VRPTW), desplegar con TLS en la nube y agregar Redis para escalar la caché, en coherencia con la formación para la vida del MEDIT [18].

## REFERENCIAS


[1] Naciones Unidas, "Objetivo 3: Garantizar una vida sana y promover el bienestar para todos en todas las edades," Agenda 2030 para el Desarrollo Sostenible, 2015.

[2] Ministerio de Salud y Protección Social, Resolución 3280 de 2018, por la cual se adoptan los lineamientos técnicos y operativos de la Ruta Integral de Atención para la Promoción y Mantenimiento de la Salud, Bogotá, 2018.

[3] Esri Colombia, "Veredas de Colombia," Datos Abiertos, servicio ArcGIS REST. [En línea]. Disponible: https://ags.esri.co/arcgis/rest/services/DatosAbiertos/VEREDAS_2016/MapServer

[4] Gobernación de Cundinamarca, "Mapa Veredal de Cundinamarca," servicio ArcGIS Feature Service. [En línea]. Disponible: https://services7.arcgis.com/lsxbLWF2l19Rmhqj/arcgis/rest/services/Mapa_Veredal_de_Cundinamarca/FeatureServer

[5] D. Luxen y C. Vetter, "Real-time routing with OpenStreetMap data," en Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems, Chicago, IL, EE. UU., 2011, pp. 513–516.

[6] OpenStreetMap Foundation, "Tile Usage Policy." [En línea]. Disponible: https://operations.osmfoundation.org/policies/tiles/

[7] D. Hardt, "The OAuth 2.0 Authorization Framework," IETF RFC 6749, oct. 2012.

[8] N. Sakimura, J. Bradley y N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients," IETF RFC 7636, sep. 2015.

[9] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, may. 2015.

[10] T. Lodderstedt, J. Bradley, A. Labunets y D. Fett, "Best Current Practice for OAuth 2.0 Security," IETF RFC 9700, ene. 2025.

[11] D. J. Rosenkrantz, R. E. Stearns y P. M. Lewis, "An analysis of several heuristics for the traveling salesman problem," SIAM J. Comput., vol. 6, no. 3, pp. 563–581, 1977.

[12] G. A. Croes, "A method for solving traveling-salesman problems," Operations Research, vol. 6, no. 6, pp. 791–812, 1958.

[13] OWASP Foundation, "OWASP Top 10:2021," 2021. [En línea]. Disponible: https://owasp.org/Top10/

[14] Congreso de la República de Colombia, Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales, Bogotá, 2012.

[15] Presidencia de la República de Colombia, Marco Ético para la Inteligencia Artificial en Colombia, Bogotá, 2021.

[16] J. Brooke, "SUS: A quick and dirty usability scale," en Usability Evaluation in Industry, P. W. Jordan et al., Eds. Londres: Taylor & Francis, 1996, pp. 189–194.

[17] K. Rodden, H. Hutchinson y X. Fu, "Measuring the user experience on a large scale: User-centered metrics for web applications," en Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI), 2010, pp. 2395–2398.

[18] Universidad de Cundinamarca, Modelo Educativo Digital Transmoderno (MEDIT), Fusagasugá, Colombia.